Quick sort with Python

I am totally new to python and I am trying to implement quick sort in it. Could someone please help me complete my code?

I do not know how to concatenate the three arrays and printing them.

```
def sort(array=[12,4,5,6,7,3,1,15]):
    less = []
    equal = []
    greater = []

    if len(array) > 1:
        pivot = array[0]
        for x in array:
            if x < pivot:
                less.append(x)
            if x == pivot:
                equal.append(x)
            if x > pivot:
                greater.append(x)
            sort(less)
            sort(pivot)
            sort(greater)
```

python algorithm sorting quicksort

edited Aug 15 '13 at 22:44

Óscar López
111k 11 111 183

asked Aug 15 '13 at 21:37

user2687481
52 1 2 6

1 You can concatenate arrays by just adding: all = less + pivot + greater Or if you just want to print them: print less + pivot + greater – kto Aug 15 '13 at 21:40

1 `pivot` is a number, so it cannot be added to `less` . You need to wrap it in a list first. – Bas Swinckels Aug 15 '13 at 21:49

11 Answers

```
def sort(array=[12,4,5,6,7,3,1,15]):
    less = []
    equal = []
    greater = []

    if len(array) > 1:
        pivot = array[0]
        for x in array:
            if x < pivot:
                less.append(x)
            if x == pivot:
                equal.append(x)
            if x > pivot:
                greater.append(x)
        # Don't forget to return something!
        return sort(less)+equal+sort(greater)  # Just use the + operator to join lists
    # Note that you want equal ^^^^^ not pivot
    else:  # You need to hande the part at the end of the recursion - when you only have
one element in your array, just return the array.
        return array
```

edited Mar 19 '14 at 11:52

Community ♦
1

answered Aug 15 '13 at 21:42

Brionius
6,126 1 7 23

@user2687481 Could you either add that to this question, or write a new question? It's too hard to tell what your code says, since the comment box removes the formatting and whitespace. – Brionius Aug 16 '13 at 4:55

1 Very pythonic and easy to read. The answer by @zangw produced triplicates in my test. This is the best answer. – Andrew Sledge May 1 '14 at 16:37

There is another concise and beautiful version

```
def qsort(arr):
```

```
    if len(arr) <= 1:
        return arr
    else:
        return qsort([x for x in arr[1:] if x<arr[0]]) + [arr[0]] + qsort([x for x in
arr[1:] if x>=arr[0]])
```

edited Apr 9 '14 at 1:52

answered Nov 28 '13 at 5:32

zangw
1,041 7 17

There are many answers to this already, but I think this approach is the most clean implementation:

```
def quicksort(arr):
    """ Quicksort a list

    :type arr: list
    :param arr: List to sort
    :returns: list -- Sorted list
    """
    if not arr:
        return []

    pivots = [x for x in arr if x == arr[0]]
    lesser = quicksort([x for x in arr if x < arr[0]])
    greater = quicksort([x for x in arr if x > arr[0]])

    return [lesser] + pivots + [greater]
```

You can of course skip storing everything in variables and return them straight away like this:

```
def quicksort(arr):
    """ Quicksort a list

    :type arr: list
    :param arr: List to sort
    :returns: list -- Sorted list
    """
    if not arr:
        return []

    return quicksort([x for x in arr if x < arr[0]]) \
        + [x for x in arr if x == arr[0]] \
        + quicksort([x for x in arr if x > arr[0]])
```

answered Aug 4 '14 at 7:57

Sebastian Dahlgren
417 6 14

1 O(N!)? is this a 'slowSort'? – Scott 混合理论 Nov 17 '14 at 4:11

I believe in the first code example it should be 'lesser' and 'greater' instead of '[lesser]' and '[greater]' - else you'll end up with nested lists instead of a flat one. – Alice Jan 15 at 19:56

Quick sort without additional memory (in place)

Usage:

```
array = [97, 200, 100, 101, 211, 107]
quicksort(array)
# array -> [97, 100, 101, 107, 200, 211]
```

```
def partition(array, begin, end):
    pivot = begin
    for i in xrange(begin+1, end+1):
        if array[i] <= array[begin]:
            pivot += 1
            array[i], array[pivot] = array[pivot], array[i]
    array[pivot], array[begin] = array[begin], array[pivot]
    return pivot


def quicksort(array, begin=0, end=None):
    if end is None:
        end = len(array) - 1
    if begin >= end:
        return
    pivot = partition(array, begin, end)
    quicksort(array, begin, pivot-1)
    quicksort(array, pivot+1, end)
```

answered Dec 13 '14 at 17:53

suquant
31 3

Simple and efficient by taking advantage of the in memory ordering. Great! – gl051 Feb 11 at 2:26

```
def quick_sort(array):
    return quick_sort([x for x in array[1:] if x < array[0]]) + [array[0]] \
        + quick_sort([x for x in array[1:] if x >= array[0]]) if array else []
```

edited Feb 4 at 16:33

p.s.w.g
87.3k 14 89 141

answered Feb 4 at 15:14

dapangmao
150 6

Or if you prefer a one-liner that also illustrates the Python equivalent of C/C++ varags, lambda expressions, and if expressions:

```
qsort = lambda x=None, *xs: [] if x is None else qsort(*[a for a in xs if a<x]) + [x] +
qsort(*[a for a in xs if a>=x])
```

edited Apr 2 '14 at 17:15

answered Apr 2 '14 at 13:30

Bruce Lucas
602 2 5

8 Lucas get in my office RIGHT NOW. What the hell is this line? B-b-b-b-but boss i-i-ts just a q-q-q-iucksort.... YOU'RE FIRED. Collect your stuff and GET OUT. – user11177 Jul 25 '14 at 7:27

functional approach:

```
def qsort(list):
    if len(list) <= 1:
        return list
    pivot = list[0]
    left = filter(lambda x: x <= pivot, list)
    right = filter(lambda x: x > pivot, list)
    return qsort(left) + (pivot,) + qsort(right)
```

answered Jun 25 '14 at 11:24

akarca
387 3 6

```
def Partition(A,p,q):
    i=p
    x=A[i]
    for j in range(p+1,q+1):
        if A[j]<=x:
            i=i+1
            tmp=A[j]
            A[j]=A[i]
            A[i]=tmp
    l=A[p]
    A[p]=A[i]
    A[i]=l
    return i

def quickSort(A,p,q):
    if p<q:
        r=Partition(A,p,q)
        quickSort(A,p,r-1)
        quickSort(A,r+1,q)
    return A
```

answered Mar 6 at 8:22

amy
11 1

Please include explanation of what your code does and how it answers the question. Especially how does it relate to the code posted in the question. Answer should give the OP and future visitors guidance on how to debug and fix their problem. Pointing out, what the idea behind your code is, greatly helps in understanding the issue and applying or modifying your solution. Stack Overflow is not a code writing service, it's a teaching and learning place. – Palec Mar 6 at 8:47

I think both answers here works ok for the list provided (which answer the original question), but would breaks if an array containing non unique values is passed. So for completeness, I would just point out the small error in each and explain how to fix them.

For example trying to sort the following array [12,4,5,6,7,3,1,15,1] (Note that 1 appears twice) with Brionius algorithm .. at some point will end up with the **less** array empty and the **equal** array with a pair of values (1,1) that can not be separated in the next iteration and the **len() > 1**...hence you'll

end up with an infinite loop

You can fix it by either returning array if *less* is empty or better by **not** calling sort in your equal array, as in zangw answer

```
def sort(array=[12,4,5,6,7,3,1,15]):
    less = []
    equal = []
    greater = []

    if len(array) > 1:
        pivot = array[0]
        for x in array:
            if x < pivot:
                less.append(x)
            if x == pivot:
                equal.append(x)
            if x > pivot:
                greater.append(x)

        # Don't forget to return something!
        return sort(less)+ equal +sort(greater)  # Just use the + operator to join lists
    # Note that you want equal ^^^^^ not pivot
    else:  # You need to hande the part at the end of the recursion - when you only have
one element in your array, just return the array.
        return array
```

The fancier solution also breaks, but for a different cause, it is missing the **return** clause in the recursion line, which will cause at some point to return None and try to append it to a list ....

To fix it just add a return to that line

```
def qsort(arr):
   if len(arr) <= 1:
      return arr
   else:
      return qsort([x for x in arr[1:] if x<arr[0]]) + [arr[0]] + qsort([x for x in
arr[1:] if x>=arr[0]])
```

answered Feb 22 '14 at 10:27

FedeN
21 2

By the way, the concise version has less performance than the long one, since it is iterating the array twice to in the list comprehensions. – FedeN Feb 24 '14 at 12:48

```
def quick_sort(list):
    if len(list) ==0:
        return []

    return  quick_sort(filter( lambda item: item < list[0],list)) + [v for v in list if
v==list[0] ]  +  quick_sort( filter( lambda item: item > list[0], list))
```

edited Mar 6 '14 at 13:33

Prashant Kumar
5,667 10 25 46

answered Mar 6 '14 at 13:16

Birger
1

```
def quicksort(items):
    if not len(items) > 1:
        return items
    items, pivot = partition(items)
    return quicksort(items[:pivot]) + [items[pivot]] + quicksort(items[pivot + 1:])


def partition(items):
    i = 1
    pivot = 0
    for j in range(1, len(items)):
        if items[j] <= items[pivot]:
            items[i], items[j] = items[j], items[i]
            i += 1
    items[i - 1], items[pivot] = items[pivot], items[i - 1]
    return items, i - 1
```

answered Dec 1 '14 at 16:24

Boubakr NOUR
736 4 7